LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

MODUL 1- PENGANTAR PEMROGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Nama : Reni Angreani

NIM : 25241043

PROGRAM STUDI PENDIDIKAN TEKNOLOGI INFORMASI

FAKULTAS SAINS, TEKNIK, DAN TERAPAN

UNIVERSITAS PENDIDIKAN MANDALIKA

TA. 2026 - 2027 (Ganjil)

Bagian 1 - Paradigma Pemrograman OOP vs Prosedural

Menurutmu, Kalau Kita Ingin Membuat Program Untuk Sistem Kampus, lebih cocok Pakai Prosedural atau OOP? Kenapa ?
Jawaban:
Menurut saya, OOP itu lebih cocok untuk sistem kampus Sistem Kampus punya banyak objek nyata seperti mahasiswa, Dosen, dan mata kuliah. Dengan OOP, setiap objek dibuat sebagai class Yang punya data dan perilaku sendiri, sehingga program lebih terstruktur. Lewat Pewarisan, Kode juga tidak Perlu di tulis ulang, dan lewat enkapsulasi, data penting seperti nilai lebih aman. Perdekatan prosedural cukup untuk program kecil, tetapi untuk sistem besar seperti kampus, OOP lebih mudah dikembangkan dan dirawat.

Bagian 2 - Class dan objek
Pada game mobile Legend, adakah Yang bisa disebut sebagai class, Jika ada apa yang menjadi attribute dan method dari class itu?
Jawaban :
Pada game mobile Legend, Yang bisa disebut class adalah hero, Karena semua hero punya data dan aksi Yang sama Jenisnya.
*Attribut (data milik hero):
- nama
- role (tank, mage, marksman, dan lain-lain)
- Hp (darah)
- attack (Kekuatan Serangan)

*Method (aksi Yang bisa dilakukan hero):
- Serang : Untuk menyerang musuh
- gunakanskill : Untuk memakai Kemampuan khusus
- bergerak : Untuk berpindah Tempat

Dari class Hero, Kita bisa membuat objek seperti Layla, Tigreal, atau Aluucard. Semuanya punya attribut dan method Yang sama, tetapi nilainya berbeda. Misalnya, Hp Tigreal lebih besar daripada Hp Layla.

Jika Sudah membuat class Pada game Mobile legend, Selanjutnya Sebutkan dua object Yang bisa dibuat dari class tersebut!

Jawaban :

Dari class Hero, dua objek Yang bisa dibuat adalah Layla dan Tigreal.

* Objek 1 : "Layla"
  - nama : "Layla"
  - Role : Marksman
  - Hp : 2500
  - Attack : 120
  - Armor : 15
* objek 2 : Tigreal
  - nama : "Tigreal"
  - Role : Tank
  - Hp : 3500
  - Armor : 25
  - Attack : 90

Kedua objek ini dibuat dari class Yang sama, Sehingga Punya attribut dan method Yang sama, Yaitu Serang(), gunakanSkill(), bergerak(), dan terimaDamage(). Perbedaanya hanya pada nilai attributnya. Angka diatas hanyalah Contoh Untuk Keperluan belajar, bukan angka asli dari game.

Praktikum 1- Class dan objek

ONLINE PYTHON

main.py +

```
1- class Hero:
2      pass
3  hero_1 = Hero()
4  hero_2 = Hero()
5  hero_1.name = "Goku"
6  hero_2.name = "Vegeta"
7  hero_1.power = 100
8  hero_2.power = 90
9  print(hero_1.name)
10 print(hero_2.name)
11 print(hero_1.__dict__)
12
```

Output :

Run | Share | Debug | $ | Command Line Argu

TERMINAL PROBLEMS VISUALIZE

```
Goku
Vegeta
{'name': 'Goku', 'power': 100}

** Process exited - Return Code: 0 **
```

Pertanyaan Praktikum 1

Jelaskan fungsi dari setiap baris pada Kode diatas, Sekalligus Jelaskan apa maksud dari perintah .__dict__!

Jawaban :

Maksud.__dict__ Adalah atribut bawaan pYthon Yang Menampilkan Seluruh atribut Sebuah objek beserta nilainya dalam bentuk dictionary.

Fungsi tiap baris :

1. Class Hero :→ Membuat class (Cetakan objek) bernama Hero.
2. Pass → Isi class dikosongkan, tidak melakukan Apa-apa.
3. Hero_1 = Hero() → Membuat objek pertama dari class hero.
4. Hero_2 = Hero() → Membuat objek Kedua dari Class hero.
5. Hero_1.name = "Goku" → Menambah atribut name berisi "Goku" Ke hero_1
6. Hero_2.name = "Vegeta" → Menambah atribut name berisi "Vegeta" ke hero_2
7. Hero_1.power = 100 → Menambahkan atribut power bernilai 100 ke hero_1.
8. Hero_2.power = 90 → Menambah atribut power bernilai 90 ke hero_2.
9. Print(Hero_1.name) → Menampilkan nama hero_1.
10. Print(Hero_2.name) → Menampilkan nama Hero_2.
11. Print(Hero_1.__dict__) → Menampilkan Semua atribut Hero_1 dalam bentuk dictionary.

Praktikum 2 - Constructor __init__()

ONLINE PYTHON

main.py +

```
1- class Mahasiswa:
2-     def __init__(self, name, nim, prodi):
3-         self.name = name
4-         self.nim = nim
5-         self.prodi = prodi
6  mhs1 = Mahasiswa("Adam", "21051214005", "Informatika")
7  mhs2 = Mahasiswa("Budi", "21051214006", "Sistem Informasi")
8  print("Nama:", mhs1.name)
9  print("NIM:", mhs1.nim)
10 print("Prodi:", mhs1.prodi)
11 print(mhs1.__dict__)
12 print()
13 print("Nama:", mhs2.name)
14 print("NIM:", mhs2.nim)
15 print("Prodi:", mhs2.prodi)
16 print(mhs2.__dict__)
```

Output :

Run | Share | Debug | $ Command Line Arguments

TERMINAL PROBLEMS VISUALIZE

```
Nama: Adam
NIM: 21051214005
Prodi: Informatika
{'name': 'Adam', 'nim': '21051214005', 'prodi': 'Informatika'}

Nama: Budi
NIM: 21051214006
Prodi: Sistem Informasi
{'name': 'Budi', 'nim': '21051214006', 'prodi': 'Sistem Informasi'}


** Process exited - Return Code: 0 **
```

Pertanyaan praktikum 2

Jelaskan Fungsi dari setiap baris pada kode diatas, Setangus Jelaskan apa yang dimaksud dengan kata kunci self dan __init__()!

Jawaban :

* self Adalah Parameter Pertama dalam Method Class Yang Merujuk Pada Objek Itu Sendiri. Dengan self, Setiap Objek Menyimpan datanya Sendiri, Sehingga data mhs1 tidak tertukar dengan data mhs2.
* __init__() Adalah Constructor, Yaitu Method Khusus Yang Otomatis dijalankan Saat Objek dibuat. Fungsinya memberi nilai awal Pada Atribut Objek.

Fungsi Tiap baris :

1. Class Mahasiswa → Membuat Class (Cetakan Objek) bernama Mahasiswa.
2. def __init__ (self, name, nim, prodi): → Membuat Constrator, Yaitu Method Yang otomatis dijalankan Saat objek dibuat, dengan Parameter name, nim, dan prodi.
3. self.name = name → Menyimpan nilai Parameter name ke Atribut name milik Objek.
4. self.nim = nim → Menyimpan nilai Parameter nim ke Atribut nim milik objek
5. self.prodi = prodi → Menyimpan nilai Parameter prodi ke atribut prodi milik objek.
6. mhs1 = Mahasiswa ("Adam", "2105121400 5", "Informatika") → Membuat objek mhs1 berisi data adam
7. mhs2 = Mahasiswa ("Budi", 2105121 4006", "Sistem Informasi") → Membuat objek mhs2 berisi data budi.
8. Print ("Nama:", mhs1.nim → Menampilkan NAMA milik mhs1.
9. Print ("NIM:", mhs1.nim → Menampilkan NIM milik mhs1.
10. Print ("prodi:", mhs1.prodi → Menampilkan prodi milik mhs1.
11. Print (mhs1.__dict__) → Menampilkan Semua atribut mhs1 dalam bentuk dictionary.
12. Print () → Mencetak baris kosong sebagai pemisah.
13. Print ("Nama:", mhs2.name → Menampilkan nama milik mhs2.
14. Print ("NIM:", mhs2.NIM → Menampilkan NIM milik mhs2.
15. Print ("prodi:", mhs2.prodi → Menampilkan prodi milik mhs2.
16. Print (mhs2.__dict__) → Menampilkan Semua atribut mhs2 dalam bentuk dictionary.

Praktikum 3 – Class dan Instance Attribut.

ONLINE PYTHON

main.py +

```python
class Mahasiswa:
    jumlah_mahasiswa = 0
    def __init__(self, name, nim, prodi):
        self.name = name
        self.nim = nim
        self.prodi = prodi
        Mahasiswa.jumlah_mahasiswa += 1
        print("Membuat Object Mahasiswa dengan nama " + self.name)
mhs1 = Mahasiswa("Adam", "21051214005", "Teknik Informatika")
print("Total Mahasiswa : " + str(mhs1.jumlah_mahasiswa))
mhs2 = Mahasiswa("Budi", "21051214006", "Teknik Industri")
print("Total Mahasiswa : " + str(mhs2.jumlah_mahasiswa))
print()
print("Total Mahasiswa : " + str(Mahasiswa.jumlah_mahasiswa))
```

Output :

Run | Share | Debug | $ | Command Line Arg

TERMINAL PROBLEMS VISUALIZE

```
Membuat Object Mahasiswa dengan nama Adam
Total Mahasiswa : 1
Membuat Object Mahasiswa dengan nama Budi
Total Mahasiswa : 2

Total Mahasiswa : 2


** Process exited - Return Code: 0 **
```

Pertanyaan Praktikum 3:

Jelaskan fungsi dari setiap baris pada kode diatas, sekaligus jelaskan mana yang menjadi Class Attribut dan mana yang menjadi Instance Attribut! Lalu apakah beda perintah print("Total Mahasiswa :"+ str(mhs1.Jumlah_Mahasiswa)) dan perintah printah print("Total Mahasiswa :"+ str(Mahasiswa.Jumlah_Mahasiswa))?

Jawaban :

* Class Attribute : Jumlah_Mahasiswa, Karena dibuat di dalam Class diluar Method dan nilainya dipakai bersama oleh semua objek.
* Instance Attribute : name, nim, dan prodi, Karena dibuat dengan self didalam _init_() dan nilainya berbeda pada tiap objek.

Perbedaan Kedua perintah print :

* mhs1.Jumlah_Mahasiswa mengakses Class Attribut melalui objek (Instance).
* Mahasiswa.Jumlah_Mahasiswa mengakses Class attribut langsung melalui nama Class.

Keduanya menghasilkan nilai yang sama karena Jumlah_Mahasiswa adalah attribut milik Class yang dipakai bersama, sehingga bukan Attribut milik objek. Cara yang lebih cepat mengaksesnya adalah lewat nama Class.

Fungsi Tiap baris :

1. Class Mahasiswa → Membuat Class bernama Mahasiswa.
2. Jumlah_Mahasiswa = 0 → Membuat attribut milik Class yang menghitung Jumlah Mahasiswa, nilai awalnya 0.
3. def_init_(self, name, nim, prodi):→ Constructor yang otomatis dijalankan saat objek dibuat.
4. self.name = name → Menyimpan nama ke objek.
5. self.nim = nim → Menyimpan NIM ke objek.
6. self.prodi = prodi → Menyimpan prodi ke objek.
7. Mahasiswa.Jumlah_Mahasiswa += → Menambah nilai Jumlah_Mahasiswa sebanyak 1 setiap ada objek baru.
8. Print("Membuat objek Mahasiswa dengan nama"+ self.name)→ Menampilkan pesan bahwa objek dengan nama tersebut di buat.

9. mhs1 = Mahasiswa("Adam", "2105121 4005", "Teknik Informatika")
   → Membuat objek mhs1.
10. print("Total Mahasiswa :" + str(mhs1. Jumlah Mahasiswa))
   → Menampilkan Jumlah Mahasiswa lewat objek mhs1.
11. mhs2 = Mahasiswa("Budi", "2105121 4006", "Teknik Industri")
   → Membuat objek mhs2.
12. Print("Total Mahasiswa :" + str(mhs2. Jumlah_Mahasiswa))
   → Menampilkan Jumlah Mahasiswa lewat objek mhs2.
13. print() → Mencetak baris kosong.
14. Print("Total Mahasiswa :" + str(Mahasiswa. Jumlah_Mahasiswa)) → Menampilkan Jumlah Mahasiswa langsung lewat class.

Praktikum 3 – Instance Method.

ONLINE PYTHON

Search

main.py +

```python
class Hero:
    # class variabel
    jumlah = 0
    def __init__(self, inputName, inputHealth, inputPower, inputArmor):
        # instance variabel
        self.name = inputName
        self.health = inputHealth
        self.power = inputPower
        self.armor = inputArmor
        Hero.jumlah += 1
    def siapa(self):
        return f"{self.name} memiliki power {self.power}"
    def healthUp(self, up):
        self.health += up
    def gethealth(self):
        return self.health
    def hitungDamage(self, musuh):
        damage = self.power - musuh.armor
        return damage
hero1 = Hero("Sniper", 100, 10, 5)
hero2 = Hero("Axe", 200, 15, 0)
print(hero1.siapa())
hero1.armor = 10
print(f"Armor {hero1.name} = {hero1.armor}")
print(f"Jumlah hero = {Hero.jumlah}")
hero1.healthUp(20)
print(f"Health {hero1.name} = {hero1.gethealth()}")
print(f"Health {hero2.name} = {hero2.gethealth()}")
print(f"Damage {hero1.name} ke {hero2.name} = {hero1.hitungDamage(hero2)}")
```

Output :

Run | Share | Debug | $ | Command line arguments

TERMINAL | PLOTS | VISUALIZE | PROBLEMS

```
Sniper memiliki power 10
Armor Sniper = 10
Jumlah hero = 2
Health Sniper = 120
Health Axe = 200
Damage Sniper ke Axe = 10


** Process exited - Return Code: 0 **
```

Fungsi Tiap baris :

1. Class Hero : → Membuat class bernama hero.
2. Jumlah = 0 → Class Variabel Untuk Menghitung Jumlah hero, nilai awalnya 0.
3. def __init__(self, InputName, Input Health, inputpower, InputArmor): Constructor Yang otomatis dijalankan Saat objek dibuat.
4. Self.Name = InputName → Menyimpan nama Ke objek.
5. Self.Health = InputHealth → Menyimpan Health Ke objek.
6. Self.power = Inputpower → Menyimpan power Ke objek.
7. Self.Armor = Input Armor → Menyimpan Armor Ke objek.
8. Hero.Jumlah += 1 → Menambah Jumlah hero Sebanyak 1 Setiap ada objek baru.
9. def Siapa(self): → Membuat Method Siapa.
10. return F"{self.name} Memiliki Power {self.power}" → Mengembalikan teks berisi nama dan power Hero.
11. def healthup(self, up): → Membuat Method healthup dengan parameter up.
12. Self.health += up → Menambah health Sebesar nilai up.
13. def gethealth(self): → Membuat Method gethealth.
14. return self.health → Mengembalikan nilai health.
15. def hitunganDamage(self, musuh): → Membuat Method hitunganDamage dengan parameter musuh.
16. damage = self.power − musuh.armor → Menghitung damage, Yaitu power Penyerang dikurangi armor musuh.
17. return damage → Mengembalikan nilai damage.
18. hero1 = Hero("Sniper", 100, 10, 5) → Membuat objek hero1.
19. Hero2 = hero("Axe", 200, 15, 0) → Membuat objek hero2.
20. Print(hero1.Siapa()) → Menampilkan hasil Method Siapa Milik hero1.
21. hero1.armor = 10 → Mengubah nilai armor Hero1 Menjadi 10.
22. print(F"Armor {hero1.name} = {hero1.armor}") → Menampilkan armor hero1.
23. Print(F"Jumlah hero = {hero.Jumlah}")) → Menampilkan Jumlah hero lewat class.

24. hero1.healthup(20) → Menambah health hero1 sebanyak 20.
25. Print(F"Health (hero1.name) = (hero1.gethealth())") → Menampilkan health hero1.
26. Print(F"health (hero2.name) = (hero2.gethealth())") → Menampilkan health hero2.
27. Print(F"Damage (hero1.name) ke (hero2.name) = (hero1.hitungDamage(hero2))") → Menampilkan damage hero1 terhadap hero2.

* Penjelasan Method tambahan yang dibuat (dibawah).
Tambahan Method yang memiliki parameter dan return.
1. Method baru HitungDamage ditulis dalam class hero, sejajar dengan method lain (setelah gethealth):
```
def hitungDamage(self, musuh):
    damage = self.power - musuh.armor return damage.
```
2. Pemanggilan Method ditulis dibagian paling bawah kode, supaya hasilnya muncul saat dijalankan:
Print(F"Damage (hero1.name ke (hero2.name) = (hero1.hitungDamage(hero2))").
3. output Tambahan
Damage Sniper ke axe = 10
* Method hitungDamage(self, musuh), memiliki parameter musuh (objek hero lawan) dan mengembalikan (return) selisih power penyerang dengan armor musuh. Karena armor Axe bernilai 0, damage Sniper sama dengan powernya, yaitu 10.

Tugas Praktikum Modul 1

Buatlah program untuk menghitung luas dan keliling dari Persegi Panjang dengan memanfaatkan OOP!

Jawaban :

Kode :

ONLINE PYTHON

main.py +

```
1- class PersegiPanjang:
2-     def __init__(self, panjang, lebar):
3-         self.panjang = panjang
4-         self.lebar = lebar
5-     def hitungLuas(self):
6-         return self.panjang * self.lebar
7-     def hitungKeliling(self):
8-         return 2 * (self.panjang + self.lebar)
9  pp1 = PersegiPanjang(10, 5)
10 print("Panjang  :", pp1.panjang)
11 print("Lebar    :", pp1.lebar)
12 print("Luas     :", pp1.hitungLuas())
13 print("Keliling :", pp1.hitungKeliling())
```

Output :

Run | Share | Debug | $ | Command Line

TERMINAL PROBLEMS VISUALIZE

```
Panjang  : 10
Lebar    : 5
Luas     : 50
Keliling : 30

** Process exited - Return Code: 0 **
```

Fungsi Tiap baris :

1. Class Persegi Panjang : → Membuat class bernama Persegi Panjang.
2. def __init__ (self, panjang, lebar): → Constractor Yang otomatis djalankan saat objek dibuat, dengan parameter Panjang dan lebar.
3. self.Panjang = panjang → Menyimpan nilai panjang ke atribut objek.
4. self.Lebar = lebar → Menyimpan nilai lebar ke atribut objek.
5. def hitungLuas (self): → Membuat Method Untuk Menghitung luas.
6. return self.panjang * self.lebar → Mengembalikan hasil Panjang dikali lebar.
7. def hitungKeliling (self) → Membuat Method Untuk Menghitung Keliling.
8. return 2 * (self.Panjang + self.lebar → Mengembalikan hasil 2 kali Jumlah Panjang dan lebar.
9. PP1 = Persegi Panjang (10, 5) → Membuat objek PP1 dengan Panjang 10 dan lebar 5.
10. Print ("Panjang :", PP1.Panjang → Menampilkan Panjang milik PP1.
11. Print ("Lebar :", PP1.Lebar → Menampilkan lebar milik PP1.
12. print ("Luas :", PP1.hitungLuas ()) → Memanggil Method hitung luas dan menampilkan hasilnya.
13. Print ("Keliling :", PP1.hitungKeliling ()) → Memanggil Method hitungKeliling dan Menampilkan hasilnya.

Penjelasan Singkat Konsep OOP Yang dipakai :

Program ini memakai class Sebagai Cetakan, objek PP1 Sebagai hasil Cetakan, atribut Panjang dan lebar Sebagai data, Serta method hitung luas dan hitung Keliling Sebagai Perilaku objek.